

vLLM 面试问题详解

并行、量化、执行链路、投机解码与 KV Cache

面向 AI Infra / 推理优化 / 高性能计算岗位

01 分布式并行 DP / TP / EP / CP MoE 两次 A2A	02 量化落地 FP8 / FP4 / AWQ / GPTQ 权重与 KV Cache	03 框架执行链 Scheduler / Executor / Worker ModelRunner
04 PD 分离 Prefill / Decode 解耦 TTFT 与 ITL	05 投机解码 Medusa / EAGLE / MTP 接受率与上限	06 内存与 KV PagedAttention Block Table / Slot Mapping

文档口径

以 vLLM V1 主线架构与公开论文为基础，重点解释“为什么这样设计、线上怎么落地、有哪些性能边界”。vLLM 更新很快，具体类名、参数名和后端支持应以所用版本为准。

整理日期：2026-08-10

使用说明与题目总览

先记住一条主线

所有题目都可以沿着“请求如何被调度 → 权重和 token 如何被切分 → KV 如何被管理 → GPU 上执行什么 kernel → 通信与计算如何重叠 → 最终优化哪个 SLO”来回答。

这份资料不是只给定义，而是按面试现场的追问路径展开：先给 30-60 秒口述结论，再补原理、数据结构、通信、性能模型和生产取舍。建议第一遍只看每节的“面试结论”和“60 秒口述版”，第二遍再啃细节。

原始问题清单

- DP、TP、EP、CP 分别解决什么问题？TP 和 EP 怎么配合？多卡部署时请求怎么切、权重怎么切？
- MoE 专家并行里的两次 All-to-All 通信怎么理解？
- 通信和计算能不能 overlap？有没有更进一步的 fusion 技术？
- Executor 为什么存在？它和 TP Worker 是什么关系？
- Prefix Cache 如何跨请求复用？
- FP8、FP4、AWQ、GPTQ 的定义和区别是什么？
- 生产环境权重量化如何落地？在线量化和离线量化怎么选？
- KV Cache 要不要量化？FP8 KV Cache 的趋势是什么？
- 量化方法和硬件后端是否绑定？框架层与算子层各负责什么？
- 从用户入口到执行层的完整链路是什么？
- Scheduler 在哪里做批处理和资源分配？
- PD 分离的价值、代价和适用场景是什么？
- Medusa、EAGLE、MTP 有什么区别？复杂并行部署下如何安排？
- Draft Model 应该独立部署、与主模型同卡，还是跟随主模型并行切分？
- 高并发下投机解码是否稳定？接受率、加速比、理论上限是多少？
- 未来优先关注原生 MTP，还是“小模型引导大模型”？
- PagedAttention 的原理是什么？Block Table、Slot Mapping 如何设计？
- KV Cache 的动态分配、复用和初始化分别在哪里完成？

目录

01 分布式并行：DP / TP / EP / CP、MoE A2A、Overlap、Executor、Prefix Cache

02 量化：FP8 / FP4 / AWQ / GPTQ、在线与离线、FP8 KV Cache、框架与 Kernel

03 框架架构与执行链路：AsyncLLM / EngineCore / Scheduler / Executor / Worker / ModelRunner、PD 分离

- 04 投机解码：Medusa / EAGLE / MTP、多卡部署、高并发稳定性、速度上限
- 05 内存管理与 KV Cache：PagedAttention、Block Table、Slot Mapping、动态分配与复用
- 06 高频问题 60 秒口述速查

参考资料

面试题答题模板：四步把系统题讲完整

步骤	面试时要回答什么
1. 定义与目标	先说它切什么、解决哪个瓶颈，以及优化 TTFT、TPOT/ITL、吞吐还是显存。
2. 数据与状态	明确请求、权重、token、KV Cache、block table 分别放在哪些 rank / worker。
3. 执行与通信	按时间线讲一次 forward：谁调度、谁执行、发生什么 collective、怎样 overlap / fusion。
4. 收益与边界	给出适用条件、负收益场景、关键监控指标，并补一句生产环境如何选。

五个常用性能指标

指标	一句话含义
TTFT	Time To First Token：从请求到首 token；主要受排队和 prefill 影响。
TPOT / ITL	每输出 token 时间 / token 间隔；主要看 decode、调度抖动和通信。
Throughput	单位时间完成的 token 或请求数；不保证尾延迟满足 SLO。
Goodput	满足延迟 SLO 的有效吞吐；PD 分离和高并发策略更应看它。
P50 / P99	中位与尾部延迟；线上“平均很快、P99 爆炸”仍然不可用。

01

分布式并行怎么配

从“切请求、切权重、切专家、切上下文”四个维度建立统一模型

Q | DP、TP、EP、CP 分别解决什么问题？

面试结论

DP 切请求和批次；TP 切每层张量/权重；EP 切 MoE 专家并动态路由 token；CP 切长上下文或 KV。四者不是同一层面的替代品，而是可组合的正交维度。

并行方式	切什么	请求如何执行	典型通信	主要解决
DP 数据并行	整个模型副本 / 一组 TP shard 被复制；请求分流到不同副本	独立请求批次	通常前端路由；MoE 场景可能还需跨 DP rank 对齐专家层	扩吞吐、扩 KV 容量、易水平伸缩
TP 张量并行	Linear / Attention / MLP 权重按维度切片	同一请求由 TP 组共同计算	All-Reduce、Reduce-Scatter、All-Gather 等	单卡放不下权重；缩短单次大矩阵计算
EP 专家并行	专家权重按 expert id 分布到不同 rank	token 由 Router 动态发送到目标专家	Dispatch A2A + Combine A2A	MoE 专家参数巨大；减少每卡专家权重
CP 上下文并行	序列 / KV Cache 沿 context 维度切分	同一长请求被多卡协作处理	Gather / Ring / 分片注意力归约	超长上下文、KV 容量或长 prefill 延迟

1.1 DP：请求级并行，不是把一个请求拆成两半

Data Parallel 的基本单位是“模型副本或模型并行组”。一个新请求会被路由到某个 DP rank，由这个 rank 管理该请求的完整生命周期和 KV Cache；另一个 DP rank 同时处理另一批请求。对于纯 Dense 模型，DP ranks 可以高度独立。vLLM 的当前文档中，每个 DP rank 对应一个独立的 core engine，并拥有自己的 TP workers。[R2]

- 权重：每个 DP rank 复制一份完整模型；若 DP rank 内还有 TP，则复制的是同构的 TP shard 组。
- 请求：一个请求通常只进入一个 DP rank，避免跨副本同步每一步 token。
- KV：每个请求的 KV 默认留在所属 DP rank；负载均衡器最好考虑 cache affinity，避免把多轮会话频繁迁移。

- MoE 例外：当专家层跨 DP $\times$ TP 组成 EP 组时，DP rank 不再完全独立，空闲 rank 也可能需要参与同步或 dummy forward。

1.2 TP：同一个请求在多张卡上共同跑一层

Tensor Parallel 将大矩阵沿输入维或输出维切分。以两层 MLP 为例，第一层常按输出通道切成 column-parallel，每个 rank 得到一部分中间激活；第二层按输入通道切成 row-parallel，最后通过 All-Reduce 或 Reduce-Scatter 聚合。Attention 中 QKV 和输出投影也可按 head 或 hidden 维切分。

面试里最容易说错的点

TP 不是“GPU0 跑请求前半段、GPU1 跑请求后半段”，那更像 Pipeline Parallel；TP 是同一层的矩阵被切片，同一个 token 在同一时刻由整组 TP rank 协作计算。

- 优点：每卡权重更少，单层矩阵计算被分摊。
- 代价：几乎每个 Transformer block 都有集合通信；TP 越大，通信占比通常越高。
- 拓扑偏好：TP 最好优先放在同机 NVLink / NVSwitch 域内，跨机 TP 往往非常吃网络时延。

1.3 EP：专家权重静态切，token 运行时动态切

Expert Parallel 专用于 MoE。专家本身在部署时静态放置在不同 rank；每个 token 经过 Router 后，根据 top-k expert id 被动态发送到对应 rank。于是 EP 的“切分对象”有两个：参数层面切专家，激活层面切 token。

- 专家权重：按 expert id 分布，例如 64 个专家 / EP16，理想情况下每 rank 常驻 4 个专家。
- token：每轮路由分布不固定，可能严重倾斜；因此容量、冗余专家、EPLB 和 load balancing 都是核心问题。
- 共享专家 / Attention / Router：通常仍按 TP 或复制方式执行，不一定跟 routed experts 使用同一种切法。

1.4 CP：把长序列与 KV 压到多卡

Context Parallel 主要针对长上下文。Prefill 时可以把 T 个新 token 分成 N 段，每张卡计算一段 Q/K/V，再通过 full-KV gather 或 ring attention 处理跨段依赖；Decode 时核心是把历史 KV 沿 context 维切到多卡，让每卡只持有一部分 KV，再聚合注意力结果。[R3]

- Prefill CP：主要降低超长 prompt 的 TTFT，并解决单卡容纳不了中间张量的问题。
- Decode CP：主要扩 KV 容量和长上下文下的 batch size。
- 代价：注意力跨卡通信增加，短上下文通常不划算。

60 秒口述版

DP 是复制模型、分请求；TP 是切每层权重，同一个请求由多卡共同算；EP 是把不同专家放到不同

卡，token 根据路由做两次 All-to-All；CP 是把长序列或 KV 沿 context 维切开。选择时先看瓶颈：吞吐用 DP，权重放不下用 TP，MoE 专家过大用 EP，超长上下文和 KV 容量用 CP。

Q | TP 和 EP 怎么配合？多卡部署时请求怎么切、权重怎么切？

面试结论

Dense/Attention 层在每个 DP rank 内做 TP；MoE 专家层可把 $DP \times TP$ 的全部 rank 组成一个 EP group。请求先按 DP 分组，进入某个 DP rank；进入专家层后，token 再按 expert id 跨 EP group 路由。

TP8 + DP2 + EP16：请求、权重与通信组的关系

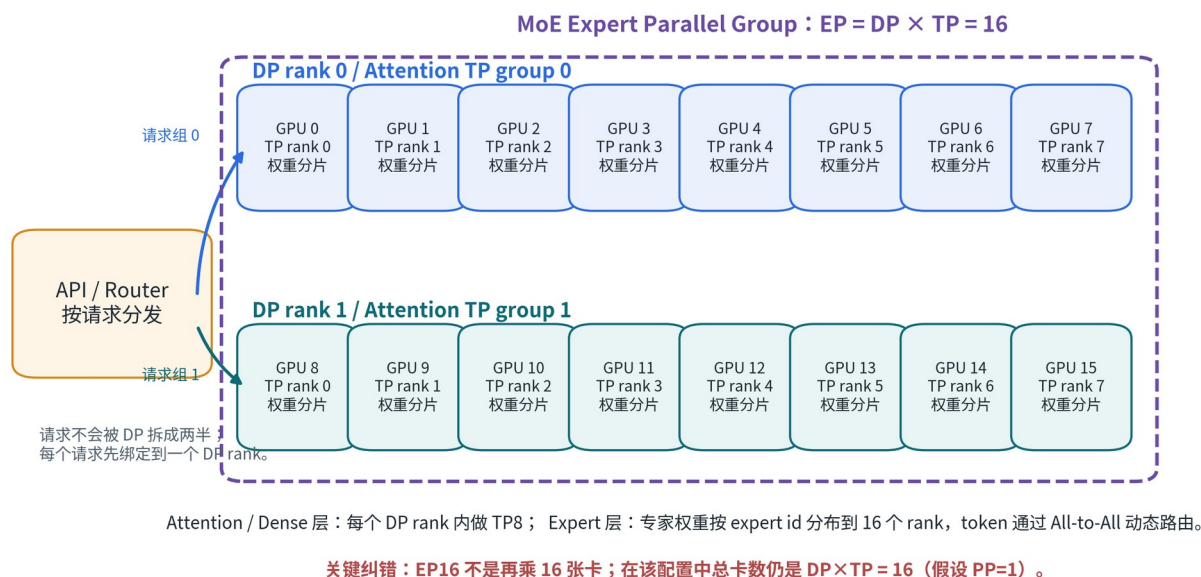


图 1：TP8 + DP2 + EP16 的典型关系。注意 EP16 覆盖已有的 16 个 rank，并非额外再乘 16。

1.5 用 TP8、DP2、EP16 逐步推一遍

1. 总卡数：若 $PP=1$ ，则总 worker 数通常是 $DP \times TP = 2 \times 8 = 16$ 。
2. Attention / Dense 层：有两个独立 TP8 组。DP rank 0 的请求在 GPU0-7 上执行；DP rank 1 的请求在 GPU8-15 上执行。
3. MoE 专家层：启用 EP 后，专家分布在全部 16 个 rank 上，因此 EP group size=16。
4. 请求进入 MoE：两个 DP rank 的 token 会先对齐一个 forward step，然后 Router 输出 expert id；token 可能被发到任意 EP rank。
5. 专家计算结束：结果通过第二次 A2A 回到源 owner rank，恢复每个请求的 token 顺序，然后继续该 DP rank 内的 Attention / Dense 计算。

工程上要补充的一句

“ $EP=DP \times TP$ ”是当前 vLLM MoE 部署里很常见的组织方式，但不同框架、不同 attention/MLA 实现、是否使用 prefill-context parallel、是否做 expert redundancy，都可能改变最终 group 构成；面试中应说明自己讲的是常见主线，而不是宣称所有框架都固定如此。[R2][R4]

1.6 权重切分与请求切分的统一视图

阶段	请求 / token 怎么切	权重 / 状态怎么切	关键性能点
入口 / DP	整个请求被路由到一个 DP rank	模型副本重复；每个 DP rank 拥有同构的 TP shards	会话亲和、队列长度、KV 可用量
Attention / Dense TP	同一请求在 TP 组内广播/分片执行	QKV、O、MLP 等矩阵按行/列或 head 切分	Collective 频率与 NVLink 拓扑
MoE EP	每个 token 根据 top-k 被复制并发往专家 rank	专家按 expert id 静态分布，可能有冗余副本	路由倾斜、A2A、Grouped GEMM
CP	同一请求的 context / KV 被切到多卡	权重可不变，主要切序列状态	长上下文、ring/gather 通信

Q | MoE 专家并行里的两次 All-to-All 怎么理解？**面试结论**

第一次 A2A 是 Dispatch：把 token 激活送到目标专家；第二次 A2A 是 Combine：把专家输出送回原请求所属 rank。中间夹着本地专家 Grouped GEMM，最后按 gate weight 聚合 top-k 结果。

MoE 专家并行：为什么通常有两次 All-to-All

第一遍 A2A 是“把 token 送到专家”；第二遍 A2A 是“把专家输出送回请求原来的 owner rank”。Top-k>1 时，同一个 token 会复制到多个专家，最后按 gate weight 聚合。

单向通信量粗略与 $T \times \text{top_k} \times \text{hidden_size} \times \text{bytes}$ 成正比，且要发生两遍；因此负载均衡、互联带宽、量化通信和计算重叠都很关键。

图 2: MoE 前向中两次 All-to-All 的角色。

1.7 Dispatch A2A：token 去找专家

- Router 对每个 token 计算 expert logits，并选择 top-k expert 与 gate weight。
- 对 token 做 permutation：按目标 rank / expert 分桶，形成连续 buffer；同时准备 source rank、token index、expert offset 等元数据。
- 执行 All-to-All，把每个桶发送到持有对应专家的 rank。Top-k>1 时，一个 token 会产生 k 份激活。
- 接收端按照 expert 分组，调用 Grouped GEMM 或 Fused MoE kernel。

1.8 Combine A2A：专家结果回到请求 owner

- 每个专家完成 MLP 后，输出仍按“专家 rank 的接收顺序”排列。
- 第二次 A2A 把结果发送回 token 的源 rank。
- 源 rank 依据 inverse permutation 恢复原 token 顺序。
- 如果 top-k>1，按 gate weight 对多个专家输出加权求和。

通信量怎么估

忽略 padding 和元数据时，单向 payload 近似为 $T \times \text{top_k} \times \text{hidden_size} \times \text{bytes_per_element}$ ；前向要做 dispatch 与 combine 两遍。实际还受负载不均、容量 padding、量化通信、跨节点比例和网络协议影响。

Q | 通信和计算能不能 overlap？有没有更进一步的 fusion？

面试结论

可以，但 overlap 需要可切分的独立工作与异步通信；fusion 则进一步消除中间读写、launch 和同步。常见做法是分块 A2A + 双缓冲 + Grouped GEMM，并融合 Router、permute、quantize、dispatch/combine、RMSNorm 或 collective。

1.9 Overlap 的基本条件

Overlap 的本质是：当某一批 token 在网络上传输时，GPU 同时计算另一批已到达 token 的专家；或者当某个 microbatch 在做 Attention / MLP 时，另一个 microbatch 做 dispatch/combine。要做到这一点，通信必须异步、数据依赖必须被切成 chunk，且要有独立 stream / event 或线程推进。

- Chunking：把 token buffer 切成多个 chunk，不等整批 A2A 完成就启动部分专家计算。
 - Double Buffer：buffer A 通信时，buffer B 计算；下一轮交换。
 - 通信流 + 计算流：用 CUDA events 表达依赖，避免全局 synchronize。
 - Microbatch / Dual Batch Overlap：两个 uBatch 交替推进，一个等待通信时另一个占用计算单元。
- vLLM 当前也有 DBO 设计。[R5]

- 跨层流水：当前层 combine 与下一层部分准备、或 attention 与 MoE 子阶段重叠，但必须严格处理残差和归一化依赖。

1.10 为什么 overlap 有时反而变慢

- 小 batch：通信/计算都太短，切块后 launch 和同步开销占比更高。
- 专家倾斜：某些 rank 收到大量 token，尾部 straggler 仍决定总时延。
- 资源竞争：通信 kernel、GEMM 同时争用 SM、HBM、L2 或 NVLink，名义重叠不等于免费。
- 过细 chunk：A2A 协议效率下降，Grouped GEMM 的 M 变小，Tensor Core 利用率变差。
- 跨节点：网络时延与拓扑不对称，必须做 node-aware routing / hierarchical A2A。

1.11 Fusion 比 overlap 更进一步在哪里

融合点	收益
Router + Top-k + Prefix-sum	减少中间 logits / index 的落盘与 kernel launch
Top-k + Permute / Quantize + Dispatch	一边重排一边低精度打包，减少 HBM 读写和网络字节
Dispatch / Combine 专用通信 kernel	把元数据、send/recv、copy 与同步路径做专门化
Grouped GEMM + 激活 / gate	在专家 MLP 内减少中间张量和多 kernel launch
All-Reduce + RMSNorm / Residual	TP collective 与后处理融合，避免额外 HBM round-trip
Megakernel / persistent kernel	把通信推进、路由、专家计算放进更大的持久 kernel，降低 CPU 和 launch 开销

回答尺度要拿捏

可以说 DeepEP、NVSHMEM、FlashInfer、Fused MoE、Megatron MoE、vLLM DBO 等生态都在做这类优化；但不要把“能异步发起”直接等同于“端到端已完全隐藏”，最终要看 trace 中通信是否落在 GEMM 的空隙里，以及 P99 是否改善。

Q | Executor 为什么存在？它和 TP Worker 是什么关系？

面试结论

Executor 是分布式执行编排抽象：对上提供一次逻辑 execute_model，对下管理一个或多个 Worker，负责进程生命周期、健康检查、权重加载、warm-up、CUDA Graph 和结果聚合。TP Worker 只是带有某个 TP rank 的单 GPU Worker。

1.12 为什么不能让 Scheduler 直接调每张 GPU

Scheduler 关注“本轮要算哪些请求、每个请求算多少 token、KV block 怎么分配”；它不应该同时承担多进程/Ray/RPC/NCCL 的进程编排细节。Executor 把单卡、多进程和集群后端统一成一套接口，使上层不必知道 worker 在本机子进程、Ray actor 还是外部 launcher 中。vLLM 的 Executor 抽象明确负责在单设备或多设备上执行模型。[R6]

组件	核心职责
Scheduler / EngineCore	请求状态、running/waiting queue、token budget、KV block、preemption、完成状态
Executor	启动/管理 worker；广播调用；健康检查；compile/warm-up/cudagraph；跨设备结果组织
Worker	一张设备一个进程；设置 device/rank；持有模型、KV cache、通信组和设备资源
ModelRunner	构造输入 tensor、block table、slot mapping、采样元数据；执行模型和 kernels

60 秒口述版

Executor 是“设备编排层”，Worker 是“单设备执行实体”。在 TP8 下通常有 8 个 Worker，每个 Worker 是一个 TP rank；Executor 把 Scheduler 的一次 SchedulerOutput 广播给这 8 个 Worker，让它们协同完成同一个 forward。Worker 内部再由 ModelRunner 准备 GPU 输入并跑模型。

Q | Prefix Cache 跨请求时怎么复用？

面试结论

对每个完整 token block 计算链式 hash，命中时让新请求的逻辑 block table 引用已有物理 KV block，并增加 refcount；分叉后的尾部再分配新块。只复用完全相同、完整 block 的前缀，主要节省 prefill，不减少后续 decode。

1.13 Prefix Cache 的核心数据结构

- Block Hash：通常包含 parent hash、当前 block token IDs，以及 LoRA、multi-modal hash、cache salt 等额外上下文，保证“同 token 但不同执行语义”不会误复用。
- Hash → Physical Block 映射：找到已计算的 KV block。
- Request → Block List：每个请求的逻辑 block table 引用物理块。
- Refcount：多个请求共享同一物理前缀块；任何一个请求结束都不能立刻释放仍被引用的块。
- Free Queue / LRU：缓存块没有活跃引用时可被保留；内存紧张时按策略淘汰并重新分配。vLLM 的 APC 设计对 full blocks、refcount、LRU 和 hash 链有明确说明。[R7]

1.14 一次跨请求复用流程

14. 请求 A 首次到达，Scheduler 为 prompt 分配物理 blocks，完成 prefill 后将完整 blocks 的 hash 放入 cache map。
15. 请求 B 到达，tokenize 后逐块计算链式 hash，找到最长连续命中的完整前缀。
16. 命中块不再计算 QKV 和 Attention，B 的 block table 直接引用 A 已有的物理 KV blocks，并增加 refcount。
17. B 从第一个未命中的 token 开始 prefill；若与 A 在中途分叉，尾部使用新块。
18. 请求结束后减少 refcount；若归零，块可留在缓存等待未来命中，也可在内存压力下被 LRU 回收。

三个边界

第一，只命中完全相同的 token 序列，字符串相似但 tokenize 不同不算；第二，通常只缓存完整 block，最后一个未满足 block 不直接共享；第三，单实例 APC 不等于跨实例共享，跨节点/PD 分离要靠 KV Connector、外部 KV store 或显式迁移。

60 秒口述版

Prefix Cache 把 prompt KV 做成内容寻址的 block cache。新请求按 token block 计算 hash，命中后 block table 直接指向已有物理 KV block 并增加引用计数，只计算未命中的尾部。它只省 prefill；如果实例不同，还需要额外 KV 传输或共享缓存层。

02

量化

先区分“数值格式、量化算法、Checkpoint 格式、运行时 Kernel”

量化问题的统一框架

FP8 / FP4 是数值格式；AWQ / GPTQ 是如何把原模型变成低比特权重的 PTQ 算法；W4A16 / W8A8 描述权重与激活精度；Marlin、CUTLASS、FlashInfer、Triton 等是具体运行时 Kernel / Backend。不要把这四层混成一件事。

Q | FP8、FP4、AWQ、GPTQ 的定义是什么？

面试结论

FP8/FP4 定义“数字怎么表示”；AWQ/GPTQ 定义“如何在尽量保精度的前提下得到低比特权重”。常见 AWQ/GPTQ 产物是 W4A16，而 FP8 常见是 W8A8 或 W8A16；实际速度取决于硬件和 kernel 是否原生支持。

概念	本质	常见形态	优势	注意点
FP8	8-bit 浮点，常见 E4M3 / E5M2；通常配 scale	W8A8、W8A16、FP8 KV	动态范围比 INT8 灵活；Hopper/更新架构可用 Tensor Core 加速	scale 粒度、溢出与校准
FP4 / NVFP4 / MXFP4	约 4-bit 浮点元素，常见 E2M1 + block scale / hierarchical scale	低比特权重与激活，尤其新架构	更低带宽与容量；硬件原生时潜力大	精度敏感、格式/布局高度后端化
AWQ	Activation-aware Weight Quantization；利用激活统计保护显著权重通道	多见 W4A16, group-wise weight-only	校准成本较低，工程生态成熟	仍需匹配高效 GEMM kernel；对模型/层有差异
GPTQ	One-shot PTQ；利用近似二阶信息逐层/逐列补偿量化误差	多见 3/4-bit weight-only	精度通常较好，配置灵活	量化过程更重；checkpoint layout 与 kernel 兼容很重要

2.1 FP8：格式本身不等于完整量化方案

FP8 常见两种编码：E4M3 用更多 mantissa bit 换精度，适合大多数权重/激活；E5M2 用更多 exponent bit 换动态范围，常用于梯度或极端范围。推理中还要决定 per-tensor、per-channel、per-token 或 per-block scale，以及 scale 是静态校准还是动态计算。vLLM 的 FP8 文档也将格式、硬件支持和量化策略分开描述。[R8]

- W8A8：权重和激活均以 FP8 参与 GEMM，通常更依赖原生 FP8 Tensor Core。
- W8A16：权重低精度存储，运行时可能反量化或使用 weight-only kernel；速度收益取决于内存瓶颈和 kernel。
- Static scale：离线统计，运行时开销小，但数据分布漂移时可能不够稳。
- Dynamic scale：每 token / 每 batch 计算 amax，适应性强，但增加 reduce + quant kernel 开销。

2.2 FP4：更低比特，必须依赖细粒度 block scale

4-bit 浮点的元素本身只能表示极少的数值，因此现代 FP4 往往不是“单独一个 E2M1 数字”，而是“E2M1 元素 + 小 block 的共享 scale + 更高层 scale”。NVFP4、MXFP4 的具体 scale 格式、block size、打包布局和硬件指令不同。面试时应强调：FP4 的性能与精度都比 FP8 更依赖硬件后端和 checkpoint layout。

2.3 AWQ：保护少量显著通道

AWQ 的观察是：并非所有权重同等重要，某些与高幅激活相乘的通道对输出特别敏感。它使用少量校准数据统计激活，通过等价缩放把显著通道变得更容易量化，然后对权重做 group-wise 低比特量化。它不需要完整反向传播，属于 PTQ。原论文强调 activation-aware 的权重保护机制。[R9]

- 常见产物：INT4 权重 + FP16/BF16 激活，即 W4A16。
- 运行时：需要支持对应 group size、zero point、packing/layout 的 kernel，例如 GEMM/GEMV 不同路径。
- 优势场景：decode 常被权重带宽限制，4-bit 权重显著减少 HBM 读量。

2.4 GPTQ：用近似二阶信息补偿误差

GPTQ 按层处理权重，利用校准激活形成的近似 Hessian 信息，量化一部分权重后把误差补偿到尚未量化的权重上，从而在 one-shot PTQ 中保持较好精度。它不是简单 round-to-nearest；核心是带有误差反馈的顺序量化。[R10]

- 常见配置：bits=4、group_size=128、对称/非对称、是否 act-order。
- 生产风险：同样叫 GPTQ 的 checkpoint 可能 packing、group、zero-point、desc_act 不同；加载成功不代表走了最快 kernel。

60 秒口述版

FP8/FP4 是低精度数值格式，决定 exponent、mantissa 和 scale 的组织；AWQ/GPTQ 是后训练量化算法，常把 BF16 权重变成 W4A16。AWQ 用激活统计保护显著通道，GPTQ 用近似二阶信息做误差补偿。最终是否加速要看硬件有没有原生指令、checkpoint layout 是否匹配高效 kernel，以及 workload 是带宽瓶颈还是计算瓶颈。

Q | 生产环境权重量化一般怎么落地？在线量化与离线量化怎么选？

面试结论

稳定生产默认优先离线量化：校准、评测、保存不可变低精度 artifact，再灰度部署；在线量化适合快速试验、模型动态加载或不想维护额外 checkpoint，但会增加启动时间、可复现性和后端耦合风险。

2.5 一条可靠的生产量化流水线

19. 定义目标：是为了“模型能放下”、降低单请求延迟、提高吞吐，还是扩大 KV / 并发？不同目标对应 W4A16、W8A8、混合精度等不同方案。
20. 建立基线：固定模型版本、tokenizer、推理框架、硬件、batch / prompt / output 分布，记录质量和性能。
21. 准备校准集：覆盖真实语言、长上下文、代码、工具调用、推理链、特殊 token；不要只用几十条普通英文。
22. 量化并做 layer sensitivity：Attention、MLP、Router、shared expert、lm_head、embedding、MTP head 可以使用不同精度或跳过。
23. 质量评测：PPL / downstream task + 真实业务回放 + 长上下文一致性 + 生成稳定性。
24. 性能评测：TTFT、TPOT、throughput、P50/P99、显存、启动时间；确认 profiler 中走的是目标低精度 kernel，而非反量化回 BF16。
25. 封装 artifact：保存 quant config、scale、group size、kernel 兼容信息、模型 hash；支持 canary 和快速 rollback。

方式	怎么做	优势	代价	推荐场景
离线量化	提前用校准数据量化并保存新 checkpoint	可重复评测；启动快；易版本化与回滚；可做复杂校准/混合精度	多一套 artifact；模型更新后需重新量化	长期稳定生产、严格质量要求、大规模多副本
在线/加载时量化	加载 BF16/FP16 权重时转换；激活可在每次 forward 动态量化	部署灵活；试验快；不必预生成低精度权重	启动慢；无/少校准；结果更依赖版本和硬件；大量副本重复转换	探索、临时模型、动态模型池、已有后端原生支持

“在线量化”不是在线学习

vLLM 当前的 online quantization 指加载 BF16/FP16 模型后在启动期转换权重，并在运行时按方案动态量化激活；它不是用户请求到来后重新训练模型。[R11]

2.6 选型经验：不要只按 bit 数排序

- Decode、低 batch、权重带宽瓶颈：W4A16 / AWQ / GPTQ 往往更有吸引力。
- Prefill 或大 batch、GEMM 计算占主导：原生 FP8 W8A8 更可能真正吃满 Tensor Core。
- MoE：专家权重占比极大，可优先量化 expert；Attention / Router / shared expert 保持更高精度，做 mixed precision。
- 老硬件：低比特 checkpoint 可能只省显存、不一定加速；甚至因反量化与不理想 kernel 变慢。
- 模型很小：量化/反量化、launch 和调度开销可能超过带宽收益。

Q | KV Cache 要不要量化？FP8 KV Cache 的趋势如何？

面试结论

当长上下文或高并发下 KV 成为显存瓶颈时，FP8 KV 很有价值，理论上相对 BF16 可把 KV 字节数约减半；但要用真实长上下文任务验证精度，并优先采用校准 scale / per-head scale。它会越来越常见，但不是所有模型和 workload 的无脑默认。

2.7 先算清楚 KV 到底占多少

对标准 Attention，一个 token 的 KV 字节数近似为：

$$\text{KV_bytes_per_token} \approx 2 \times \text{num_layers} \times \text{num_kv_heads} \times \text{head_dim} \times \text{bytes_per_element}$$

前面的 2 分别对应 K 和 V

示例：32 层、8 个 KV heads、head_dim=128。BF16 每元素 2 bytes，则每 token 约 128 KiB；32K context 约 4 GiB。换成 FP8 后约 64 KiB/token、32K context 约 2 GiB。这里只计算单条序列的 KV 主体，未计 block 元数据、对齐和工作区。

2.8 FP8 KV Cache 的收益与代价

维度	影响
显存 / 并发	KV 主体约减半，可放更长 context 或更多并发请求
带宽	Attention 读取 K/V 字节减少；若 backend 可直接在 FP8 域计算，收益更完整
额外开销	写 KV 时 quantize、读时 dequantize，或 Q/K/V 同域转换；scale 读写也有成本
精度	长上下文、检索、代码和 reasoning 对误差可能更敏感；需校准和层级豁免

维度	影响
Scale 粒度	per-tensor 最简单；per-head 更细但元数据/支持更复杂

vLLM 当前支持 FP8 KV 的 per-tensor 和 per-attention-head scale；文档推荐用校准数据估计 scale，某些 backend 可在 FP8 域完成 attention，并允许跳过敏感层。[R12]

什么时候不急着量化 KV

短上下文、低并发、模型权重已经占满主要显存，或业务对极细微生成差异非常敏感时，先量化权重或优化 batching 可能更划算。KV 量化收益依赖“KV 是否真的是当前瓶颈”。

60 秒口述版

我会先用公式估算 KV 占比。如果长上下文或高并发下 KV 是主要显存和带宽瓶颈，就优先尝试 FP8 KV，理论上从 BF16 到 FP8 可把 KV 主体约减半。生产上会用代表性数据校准 scale，最好按 head，重点回归长上下文检索、代码和 reasoning；短上下文或精度特别敏感时不一定开启。

Q | 不同量化方法跟硬件后端是否绑定？框架层和算子层各负责什么？

面试结论

算法概念可以跨硬件，但高性能实现强绑定后端：数据格式、packing、scale 粒度、tile、Tensor Core 指令和 fused kernel 必须匹配。框架层负责识别配置与选择路径；算子层负责真正低精度计算和数据布局。

2.9 为什么“模型能加载”不等于“量化有效”

同一个 AWQ/GPTQ/FP8 checkpoint，在不同 GPU、不同 driver、不同 attention backend 上可能走完全不同路径：原生低精度 GEMM、专用 weight-only kernel、先反量化再 BF16 GEMM，甚至 fallback 到慢实现。性能绑定的不是算法名字，而是“格式 × layout × kernel × hardware × workload”的组合。vLLM 的量化支持矩阵也明确按硬件平台区分。[R13]

层次	职责
框架 / Engine 层	解析 checkpoint 与 quant config；校验 hardware capability；决定哪些 layer 量化；加载/repack 权重；保存 scale；选择 quant method 与 backend；管理 fallback。
模型层 / Module 层	把 Linear、MoE、Attention 替换为带 QuantMethod 的模块；为不同 layer 返回不同精度策略。
算子 / Kernel 层	实现 packed weight 读取、scale 应用、activation quant、GEMM/GEMV、MoE grouped GEMM、FP8 Attention、fused epilogue；处理 alignment、tile 和通信。

层次	职责
编译 / 图优化层	融合 quantize + matmul、RMSNorm + quant、attention quant、collective + norm；减少中间张量和 launch。

2.10 面试里可以举的一个故障例子

“我加载了 W4A16 模型，显存下降但速度没变甚至变慢。” 排查顺序应是：是否真的走 Marlin/CUTLASS 等目标 kernel；shape 是否满足 tile/alignment；batch 是 GEMV 还是 GEMM；是否每层动态反量化；group size 与 checkpoint packing 是否匹配；MoE 是否发生大量小 M 专家 GEMM；CPU preprocessing 或 graph break 是否变成新瓶颈。

03

框架架构和执行链路

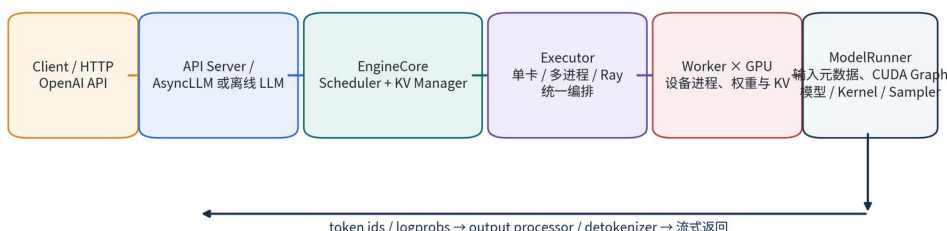
把“谁调度、谁编排、谁持有 GPU 状态、谁真正跑 Kernel”说清楚

Q | 从用户入口到执行层的完整链路是什么？

面试结论

在线链路可概括为：Client/API Server → AsyncLLM/Engine Client → EngineCore (Scheduler + KV Cache Manager) → Executor → Worker → ModelRunner → Model/Kernels/Sampler → Output Processor/Detokenizer → 流式返回。离线 LLM 省去 HTTP 层，但核心执行链相同。

vLLM V1 主线：从请求入口到 GPU Kernel 的执行链路



最重要的层次关系：Scheduler 决定“本轮算谁、算几个 token”；Executor 决定“调用哪些设备进程”；Worker 持有单设备状态；ModelRunner 把 SchedulerOutput 转成 GPU 输入并真正执行模型。

类名和进程边界会随版本演进，但“入口层 - 调度/KV 管理 - 分布式执行 - 每卡运行器 - Kernel”这条职责链基本稳定。

图 3: vLLM V1 主线职责链。具体类名会演进，职责边界更重要。

3.1 入口层：API Server、AsyncLLM 与 LLM

- API Server：处理 HTTP/OpenAI 兼容协议、参数校验、鉴权/限流、tokenization、多模态输入、取消请求、流式输出。
- AsyncLLM：面向在线并发的异步接口，把请求送入引擎并异步拉取输出；适合 server。
- LLM：更偏离线同步批处理接口，脚本直接调用；没有 HTTP 网络层，但仍使用底层 EngineCore / Scheduler / Worker。

vLLM 的架构文档将输入处理、调度、模型执行和输出处理拆成不同职责，并说明 worker 通常一张加速器一个进程，每个 worker 有一个 model runner。[R1]

3.2 EngineCore：请求状态机与执行循环中心

EngineCore 接收已经预处理的请求，维护 request state，循环执行 schedule → execute_model → update_from_output。它通常包含 Scheduler、KV Cache Manager、结构化输出状态、Spec Decode 状态等。对于 DP 部署，每个 DP rank 可对应独立 EngineCore。

```
while has_requests:
    scheduler_output = scheduler.schedule()
    model_output = executor.execute_model(scheduler_output)
    engine_outputs = scheduler.update_from_output(
        scheduler_output, model_output
    )
    publish(engine_outputs)
```

3.3 Executor、Worker、ModelRunner 再区分一次

- Executor：跨设备编排。一个 SchedulerOutput 对应一次逻辑执行，Executor 把它发给所有参与 rank。
- Worker：单设备进程，初始化 distributed group、device、model、KV cache；拥有 rank/local_rank。
- ModelRunner：单设备执行器，维护 persistent batch，构造 attention metadata、block table、slot mapping，执行 model.forward、sampler、CUDA Graph。
- Model：真正的 torch.nn.Module 和 quantized modules；内部调用 Attention、Linear、MoE 等 kernels。

一句话记层次

Scheduler 选工作；Executor 分发工作；Worker 管设备；ModelRunner 组织这张卡的一次 forward；Kernel 完成具体数学。

Q | Scheduler 在哪里做批处理和资源分配？

面试结论

Scheduler 在 EngineCore 内每个迭代步做 continuous batching：从 running / waiting 请求中按 token budget 和 KV block budget 选请求，决定每个请求本轮算多少 token，并调用 KV Cache Manager 做命中、分配、追加、抢占或释放。

3.4 Continuous Batching 不是“一次组批后跑到底”

传统静态 batching 在一批请求全部结束前很难插入新请求；LLM 每条输出长度不同，会造成大量尾部空槽。Continuous batching 则每次 forward 后重新组批：完成的请求退出，新请求进入，prefill 和 decode 可在 token 级别共同受预算约束。

3.5 Scheduler 每轮主要做什么

26. 扫描 running 请求：优先给正在 decode 的请求分配新 token，或按策略处理 chunked prefill / speculative tokens。
27. 检查 token budget：例如 max_num_batched_tokens，决定每个请求本轮计算多少 token。
28. 检查 KV budget：确认所需 blocks 是否可分配；尝试 prefix cache / remote cache 命中。
29. 空间不足时做 preemption：暂停/重算/交换某些请求，释放 blocks。
30. 从 waiting 队列纳入新请求，直到 token、sequence、encoder 或 KV 预算用尽。
31. 生成 SchedulerOutput：请求 IDs、num_scheduled_tokens、block IDs、cached tokens、spec tokens、encoder inputs 等。

资源维度	Scheduler 的决策
Token Budget	本轮 GPU 能处理多少 token；影响 prefill chunk、decode batch 和 verify 扩张
Sequence Budget	同时活跃请求数 / max_num_seqs；影响元数据和 CUDA Graph shape
KV Block Budget	可用物理 blocks；决定能否接入新请求、追加 decode、是否 preempt
公平性 / 优先级	FCFS、priority、SLO、租户隔离；防止长 prompt 或大输出长期占用
缓存命中	prefix / external KV 命中会改变 num_computed_tokens 和实际需调度 token

为什么 Scheduler 是性能核心

同一个 kernel，在不同 schedule 下会呈现完全不同的 TTFT、TPOT 和吞吐。过大 prefill chunk 会干扰 decode；过小又浪费 GEMM；speculative K 太大使 verify batch 膨胀；KV block 分配不合理会频繁 preempt。

60 秒口述版

Scheduler 位于 EngineCore 的主循环里，不是在 API Server。它每个 step 都重新做 continuous batching：根据 running/waiting 队列、token budget、max sequences 和 KV blocks，决定每个请求本轮算多少 token；KV Cache Manager 负责查 prefix hit、分配/释放 block。然后 SchedulerOutput 交给 Executor 和 Workers 执行。

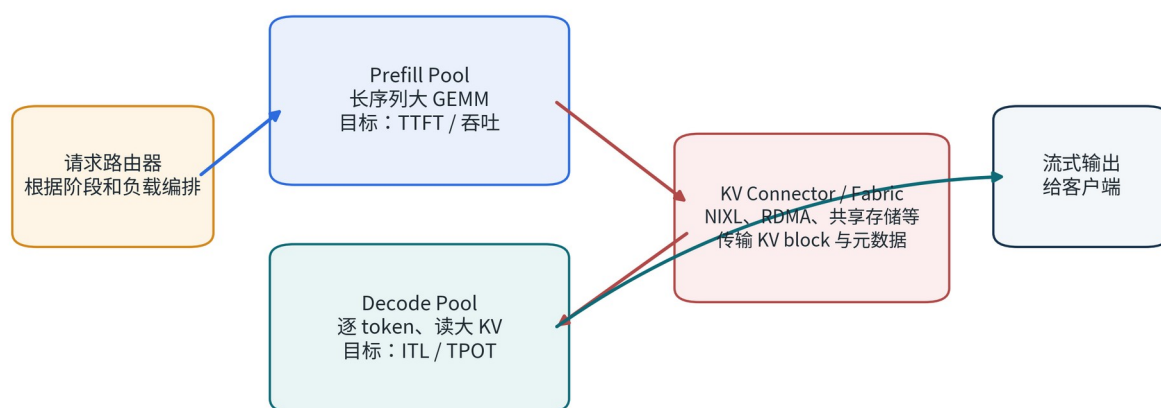
Q | PD 分离的价值和适用场景是什么？

面试结论

Prefill 适合大 batch / 大 GEMM，关注 TTFT；Decode 每步 token 少但反复读权重和 KV，关注 ITL/TPOT。PD 分离把两类资源池独立扩缩容和调优，避免长 prefill 干扰 decode；代价是 KV 传输与更复杂的路由、排队和故障处理。

PD 分离：Prefill 与 Decode 为什么要拆开部署

价值：阶段隔离、独立扩缩容、不同并行策略、控制 tail ITL



代价：KV 传输、额外排队与网络跳数、路由/回压/故障处理复杂；并不天然提升原始吞吐，重点是 SLO 下的 goodput。

图 4：PD 分离的基本数据流。

3.6 Prefill 与 Decode 的性能画像不同

阶段	计算特征	核心 SLO	常见优化
Prefill	一次处理大量 prompt tokens；大 GEMM，算术强度较高	TTFT、prefill throughput	更大 batch、TP/CP、chunk size、FlashAttention
Decode	每请求每步通常 1 token；权重/KV 读取频繁，常 memory-bound	ITL / TPOT、P99、并发	高 KV 容量、DP、低延迟通信、CUDA Graph、spec decode

3.7 PD 分离带来的价值

- 隔离干扰：长 prefill 不再插入 decode batch，tail ITL 更可控。
- 独立扩缩容：prompt 激增时扩 P pool；长输出激增时扩 D pool。
- 不同并行策略：P 可用更大 TP/CP、D 可用更多 DP 或不同 quant/KV 配置。
- 硬件异构：P 使用算力更强设备，D 使用内存带宽/容量更合适设备。

- SLO 路由：按 prompt length、预估 output、租户等级选择不同 P/D pool。

3.8 PD 分离的代价与判断条件

- KV 传输量大：要传每层 K/V blocks，长 prompt 时可达 GB 级；网络带宽和零拷贝路径决定成败。
- 额外排队与跳数：P 完成后等待 D slot，可能抵消 TTFT 改善。
- 缓存一致性：block IDs、layout、dtype、TP mapping 必须兼容；失败恢复复杂。
- 路由与回压：P 生产 KV 的速度不能长期超过 D 消费能力。
- 低负载/短 prompt：单体引擎更简单，PD 的网络和调度开销可能不值。

vLLM 的 disaggregated prefill 文档强调，它允许分别调 TTFT 与 ITL、控制 tail ITL，但当前文档明确提示“并不自动提升吞吐”；应关注 SLO 约束下的 goodput。[R14] DistServe 等系统论文则展示了在特定集群和 SLO 下的资源解耦收益。[R15]

60 秒口述版

PD 分离适合长 prompt、高负载、同时严格要求 TTFT 和 ITL 的在线服务。Prefill 算大 GEMM，Decode 每步读大量权重和 KV，两者最优并行和资源比例不同。拆开后能独立扩缩容并隔离 tail latency，但要付出 KV 传输和复杂调度成本，所以我会用端到端 goodput 和 P99 判断，不会说它天然提升吞吐。

04

投机解码

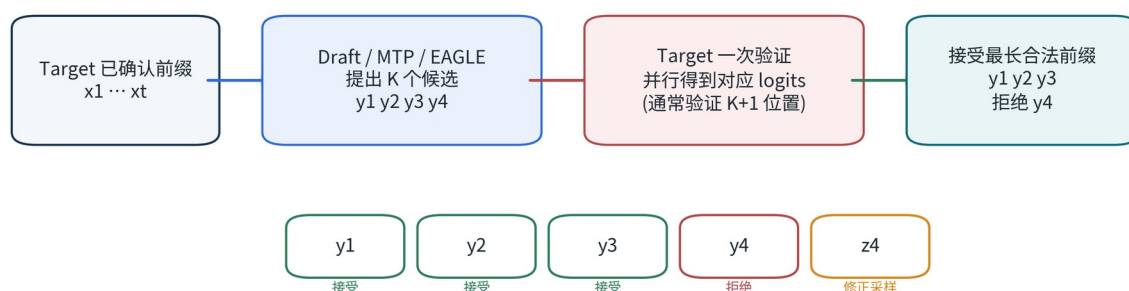
把“候选从哪来、Target 如何验证、部署如何放置、何时会负收益”说透

Q | 投机解码的基本原理是什么？

面试结论

先用便宜 proposer 提出 K 个未来 token，再让 target 一次 forward 并行验证这些位置；接受最长合法前缀，拒绝处按修正规则采样。无损算法保持与普通自回归解码相同的输出分布。

投机解码：一次“草稿 + 验证 + 接受前缀”的完整轮次



无损投机解码的目标不是“近似大模型”，而是用廉价 proposer 减少昂贵 target forward 次数；接受/拒绝规则保证最终采样分布与普通自回归解码一致。

速度取决于：平均每轮提交 token 数、draft 开销、verify(K+1) 开销、接受率、批大小和缓存/元数据开销。K 越大并不一定越快。

图 5：一次投机解码轮次。

4.1 为什么能加速

普通 decode 每产生 1 个 token 都要执行一次大模型 forward。投机解码把大模型一次 forward 的“序列维并行能力”利用起来：虽然验证 K+1 个位置比验证 1 个位置更贵，但通常远小于连续做 K+1 次 target forward。关键前提是 proposer 足够便宜且候选接受率高。

经典 speculative sampling 通过接受/拒绝与修正分布保证无损；原论文在特定模型和硬件上报告过 2-3× 级别收益，但这些数字不能直接当作线上承诺。[R16]

Q | Medusa、EAGLE、MTP 区别在哪？

面试结论

Medusa 给 target 增加多个未来 token heads，常用树状候选与 tree attention；EAGLE 用轻量 draft 模块结合 target hidden features 进行更强的自回归提议；MTP 是 target 原生训练出的多 token prediction 模块，不需要外部 draft checkpoint。

方案	候选来源	部署形态	优点	限制
Medusa	在 target hidden state 上增加多个 decoding heads，并行预测不同未来位置；常构造候选树	附着在主模型上，额外 heads 较轻	结构直观、候选可并行	多步相关性有限；树验证和 head 训练/适配
EAGLE	轻量 draft 模块利用 target 隐状态/特征和 token 逐步提出候选；不同版本细节有差异	通常与 target 紧耦合，需兼容的 EAGLE checkpoint	接受率高、通用性较强	需要额外模型/隐藏状态接口；部署更复杂
MTP	Target 在训练时原生包含多 token prediction 层/模块	权重随 target checkpoint，一般同进程同设备执行	配置简单、无外部 draft；共享表示	只适用于原生支持的模型家族；spec depth 受实现约束
独立 Draft Model	更小的普通 LM 自回归生成 K 个 token	可同卡、同节点或独立资源	模型覆盖广、易理解	需要额外 KV/权重；远程部署可能被通信吃掉

vLLM 当前将 EAGLE、MTP、draft model 等列为 model-based speculation；其 MTP 文档明确指出 MTP 使用 target 原生能力，不需要单独 draft model。[R17][R18] Medusa 与 EAGLE 原论文分别报告了各自模型/任务上的加速，但应以自己的 batch、sampling 和硬件复测。[R19][R20]

60 秒口述版

Medusa 是给大模型加多个未来 token heads，常配候选树；EAGLE 是一个轻量、利用大模型 hidden feature 的更强 draft 模块；MTP 是大模型训练时原生带的多 token prediction 层，不需要外部 draft。部署上 Medusa/MTP 最合同主模型共置；EAGLE/小 draft 也尽量同节点甚至同卡，避免远程 RPC 把收益吃掉。

Q | 多卡、多机、TP8 DP2 EP16 下怎么安排投机解码？

面试结论

Target 的请求仍先绑定一个 DP rank；Attention 验证走该 rank 的 TP8，MoE 专家层走 EP16。Medusa/MTP heads 跟随 target 的权重切分；EAGLE/小 draft 最好按 DP replica 共置并复制，必要时自身也做 TP，但不要为了省一点显存把 draft 放成高时延远程服务。

4.2 以 TP8 DP2 EP16 为例

32. 请求路由：请求 A 绑定 DP rank 0，后续 target verification 主要由 GPU0-7 的 TP8 组发起；请求 B 可在 DP rank 1。
33. Target MoE：验证经过专家层时，两个 DP ranks 的 token 仍可能跨 EP16 做 dispatch/combine。
34. MTP / Medusa：附加 heads 或 MTP layers 是 target checkpoint 的一部分，应与 target 采用一致 TP/quant/graph 路径；通常不独立成服务。
35. EAGLE / 小 Draft：最常见做法是每个 DP replica 配一个 draft 副本，使请求不跨 DP 迁移；draft 很小时可在同 GPU 组内复制或仅在每组一个 rank 执行。
36. Draft 也很大：可为 draft 设置较小 TP，或单独占用少量 GPU，但必须评估 hidden states / tokens / KV 在 target-draft 之间的传输、同步和 pipeline bubble。

4.3 Draft Model 应该放在哪里

放置方式	优点	代价	适用
同 GPU / 同进程	最低通信时延；易共享 hidden state；无需 RPC	占用 target 显存和计算资源；可能影响并发	MTP、Medusa、很小 EAGLE/MLP
同节点独立 GPU	资源隔离；NVLink/PCIe 较快；可 pipeline	需要跨 GPU 传 token/hidden/KV；调度更复杂	draft 不小且 target GPU 显存紧张
独立远程服务	可独立扩缩、共享 draft fleet	网络往返、排队、状态/KV 传输极易抵消收益	只有超大规模、强网络和精细批处理时考虑
跟随 TP 切分	draft 本身也能放下并利用多卡	draft 的 TP collective 可能让“便宜 proposer”不再便宜	draft 仍较大且单卡放不下

部署原则

投机解码优化的是“每个已提交 token 的 target 成本”。如果 proposal 的 RPC、排队、TP collective、hidden state copy 或 draft KV 管理变贵，就会把最核心的收益吞掉。因此默认共置，只有 profiling 证明资源争用更严重时才拆。

Q | 线上高并发时投机解码稳不稳定？接受率、加速比、理论上限大概多少？

面试结论

中低 QPS、decode memory-bound 时最容易获益；高 QPS 下 verify 一次处理 K+1 倍 token，可能把 target 从带宽瓶颈推到计算瓶颈，收益缩小甚至负收益。必须动态调 K 或按队列/批大小关闭。

4.4 接受率的三个指标

- Token Acceptance Rate: 被接受的 draft tokens / 提议 tokens。适合看 proposer 质量。
- Accepted Length: 每轮连续接受的前缀长度 A；比单 token acceptance 更直接决定 forward 减少量。
- Effective Tokens per Target Forward: 每次 target verify 最终提交多少 token，通常约 A+1，但具体取决于算法和 EOS/修正规则。

4.5 一个简化期望公式

若每个 draft token 独立以概率 α 被接受，连续接受前缀长度的期望可近似为：

$$\begin{aligned} E[A] &= \alpha + \alpha^2 + \dots + \alpha^K \\ &= \alpha \times (1 - \alpha^K) / (1 - \alpha) \end{aligned}$$

每轮提交 token 数大致 $\approx E[A] + 1$

设置	E[接受 draft 数]	约每轮提交
K=4, $\alpha=0.80$	2.36	约 3.36
K=4, $\alpha=0.90$	3.10	约 4.10
K=8, $\alpha=0.80$	3.33	约 4.33
K=8, $\alpha=0.90$	5.13	约 6.13

这个表说明 K 增大存在明显边际递减，尤其当 α 不够高时。更重要的是，K 变大还会增大 draft 时间、verify token 数、临时 KV 和 attention metadata。

4.6 更接近工程的速度模型

$$\text{speedup} \approx E[\text{committed_tokens}] \times T_{\text{target}}(1) / (T_{\text{draft}}(K) + T_{\text{verify}}(K+1) + T_{\text{accept/reject}} + T_{\text{sync}})$$

- 理想上限：当接受率=1、draft/verify 增量开销 ≈ 0 时，最多接近 K+1；现实中不可能完全满足。
- 高 batch：T_verify(K+1) 不再近似 T_target(1)，可能显著增长。
- 采样温度高：draft 与 target 分布更难一致，接受率往往下降。
- 长 context：verify attention 读更大 KV；但减少 target 步数的价值也可能更高，需要实测。
- MoE：K+1 token 会改变路由和 expert batch，可能改善 Grouped GEMM，也可能增加 A2A。

4.7 高并发稳定性的工程策略

- Dynamic Speculation: 根据队列深度、batch tokens、最近 acceptance、P99 TPOT 动态调 K。
- Admission Control: 只对长输出、低温度、命中某类模型/请求启用。
- Fallback: 高峰期切到 n-gram/suffix 或直接关闭 model-based proposer。
- 独立指标: draft latency、verify latency、accept length、rejection rate、spec KV bytes、每轮 committed tokens。
- 按 SLO 评估: 不能只看单请求 latency; 要同时看吞吐、P50/P99、GPU 利用率和 OOM/preemption。

vLLM 当前文档将 speculative decoding 定位为中低 QPS、memory-bound workload 的典型优化, 并提供 dynamic speculation; 同时也指出不同方法在高 QPS 下的收益依赖实际模型和流量。[R17]

60 秒口述版

投机解码的理论上限接近 $K+1$, 但真实速度由接受长度、draft 成本和 verify($K+1$) 成本共同决定。高并发时 verify batch 会膨胀, target 可能从 memory-bound 变成 compute-bound, 所以必须根据 queue depth 和 acceptance 动态调 K, 必要时关闭。线上我会看 accepted length、committed tokens/target forward、P99 TPOT 和吞吐, 不只看 acceptance rate。

Q | 未来优先关注原生 MTP, 还是“小模型引导大模型”?

面试结论

原生 MTP 是模型支持时的首选默认路线: 共置、配置简单、共享表示; EAGLE/小 draft 仍是覆盖更多模型、追求更高接受率和可独立迭代的重要路线。真正长期关键是动态调度、可观测性和与 serving stack 的深度融合, 而不是只押一个算法名字。

- 优先 MTP: 目标模型原生提供、框架支持成熟、质量/速度已验证。
- 优先 EAGLE / draft: 模型没有 MTP, 或兼容 draft 在该任务上显著提高 acceptance, 且共置成本可控。
- 高 QPS 兜底: n-gram / suffix 这类轻 proposer 没有额外大模型工作, 可能更稳。
- 面向训练-推理协同: 未来模型很可能在预训练阶段就考虑 MTP / block prediction, 但 serving 仍需根据负载动态决定是否使用。

05

内存管理与 KV Cache

理解 PagedAttention 的关键不是 kernel 细节，而是逻辑序列到物理块的映射与生命周期

Q | PagedAttention 的原理是什么？Block Table、Slot Mapping 如何设计？

面试结论

把 GPU KV Cache 预分配成固定 token 数的物理 blocks；每个请求只维护逻辑 block 列表。Block Table 将逻辑块映射到任意物理块，Attention 据此读取历史 KV；Slot Mapping 给本轮新 token 指定具体写入槽位。

PagedAttention：逻辑序列如何映射到离散物理 KV Blocks

逻辑请求视角

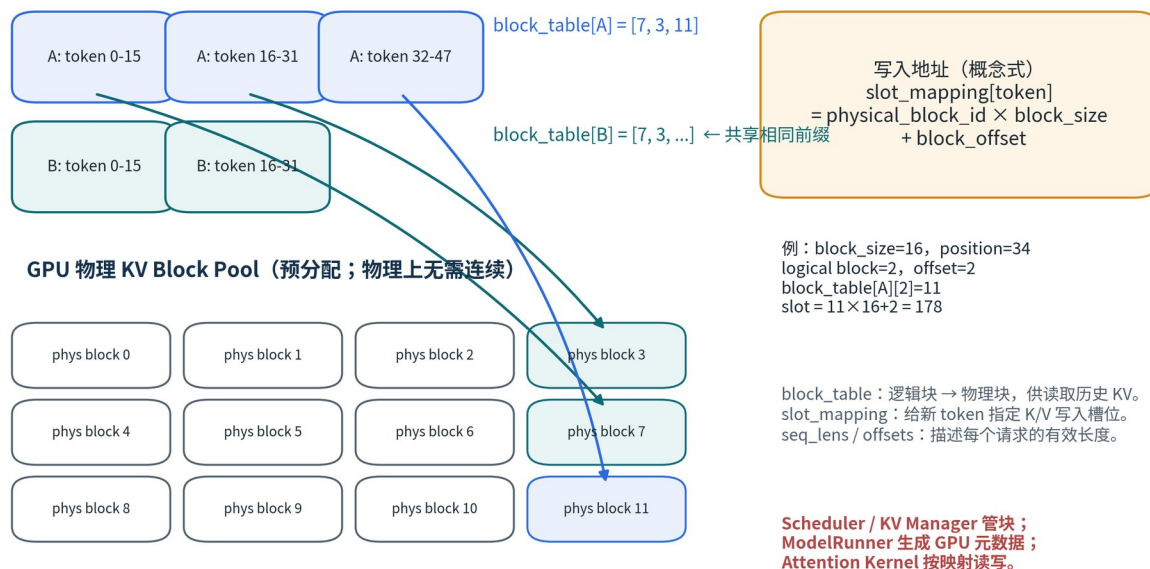


图 6：Paged KV 的逻辑到物理映射，以及跨请求共享前缀。

5.1 为什么不用每个请求一段连续大数组

请求长度事先未知且持续增长。若为每个请求按 max context 预留连续空间，会严重浪费显存；若每步 realloc/copy，又会产生碎片和昂贵搬迁。PagedAttention 借鉴虚拟内存，把逻辑连续的 token 序列

映射到物理上离散的固定大小 blocks，使 Scheduler 可以按需追加、回收和共享。vLLM 原始论文和当前 paged KV kernel 文档都围绕这一思想。[R21][R22]

5.2 三个核心数据结构

结构	形态	作用
Physical KV Pool	[num_blocks, block_size, ...] 的 K/V tensors	真正存储每层每个 token 的 K/V；启动时大块预分配
Block Table	[num_reqs, max_blocks_per_req] 的物理 block ids	读路径：logical block index → physical block id
Slot Mapping	[num_scheduled_tokens] 的 slot ids	写路径：本轮每个 token 的 K/V 写到哪个物理位置
Seq Lens / Query Offsets	每请求长度、prefill/decode 分段、cu_seqlens 等	告诉 Attention 每个 query 需要读取多少历史 KV
RefCount / Free Queue	每个物理 block 的引用计数与空闲链表	支持共享前缀、fork、释放和 LRU 淘汰

5.3 Block Table：逻辑连续，物理可以离散

假设 block_size=16，请求 A 的 block_table=[7,3,11]。那么 token 0-15 的 KV 在 physical block 7，16-31 在 block 3，32-47 在 block 11。Attention kernel 处理某个 query 时，根据 token position 算出 logical block index 和 offset，再通过 block table 找到物理地址。

- 优势：新增 token 只需再分配任意空闲 block，无需搬迁前面的 KV。
- 代价：多一次间接寻址；需要把 block table / metadata 高效传到 GPU。
- Block size 权衡：小 block 减少内部碎片和提升 prefix 命中粒度，但元数据与调度开销更高；大 block 相反。

5.4 Slot Mapping：本轮新 K/V 写到哪里

Slot Mapping 通常是“每个实际计算 token 对应一个线性 slot”。概念上：slot = physical_block_id × block_size + offset。ModelRunner 根据 Scheduler 分配的 blocks、每个请求当前长度和本轮 token 数，生成 slot_mapping；Attention / cache kernel 按它把新 K/V scatter 到 paged pool。具体 backend 可能有更复杂的 layout，但职责不变。

读写职责不要反

Block Table 主要解决“Attention 读历史 KV”；Slot Mapping 主要解决“本轮新 token 把 K/V 写到哪个槽”。两者经常同时作为 attention metadata，但语义不同。

Q | KV Cache 的动态分配、复用和初始化位置怎么做？

面试结论

物理 KV tensors 在 Worker/ModelRunner 初始化时按可用显存一次性创建；Scheduler 侧 KV Cache Manager 在 CPU 上管理 block IDs、refcount、free queue 和 prefix hash；ModelRunner 每步把分配结果转成 GPU block table/slot mapping，Kernel 直接读写预分配 pool。

5.5 初始化：先建池，不按请求 cudaMalloc

37. Worker 初始化设备、加载模型并做 memory profiling，估算模型权重、CUDA Graph、workspace 后剩余多少显存可给 KV。
38. 根据 block_size、layer 数、head 数和 dtype 计算每个 block 字节数，得到 num_gpu_blocks。
39. ModelRunner / attention backend 为每层创建 K/V cache tensors，形成物理 block pool。
40. Scheduler 侧创建同样数量的 block metadata、free queue 和 KV Cache Manager，但不存实际 K/V 数据。

5.6 每个请求的动态生命周期

41. 入队：tokenize 后检查 prefix cache / remote KV，确定 num_computed_tokens。
42. Prefill 调度：KV Cache Manager 为未命中 token 预留 blocks，SchedulerOutput 携带 block IDs。
43. 执行：ModelRunner 更新 persistent batch、block table、slot mapping；kernel 将新 K/V 写入 pool。
44. Decode：每新增 token 使用当前尾 block 的下一个 offset；满 block 后再取一个 free block。
45. Fork / Beam / 多轮前缀：多个序列可共享已有 blocks，refcount++；分叉后只给新尾部分配块，类似 copy-on-write。
46. 完成 / 取消 / Preempt：refcount--；归零后回 free queue，若启用 prefix cache 可暂时保留 hash 直到被淘汰。

5.7 谁负责什么

组件	KV 相关职责
Scheduler	决定本轮哪些请求、多少 token；触发 allocate/free/preempt；维护 request state。
KV Cache Manager (CPU)	物理 block id 分配、free queue、refcount、prefix hash、LRU、remote connector 状态。
Executor / Worker	把调度结果送到各设备；保持各 TP rank 的 block 映射一致。

组件	KV 相关职责
ModelRunner	将 block IDs 转为固定 GPU tensor / metadata；更新 persistent batch；准备 slot mapping。
Attention / Cache Kernel	按 slot 写新 K/V，按 block table 读取历史 K/V，执行 attention。

5.8 PagedAttention 的性能代价

- 间接寻址和不连续内存：kernel 需要根据 block table gather，可能影响 coalescing。
- Metadata：block table、slot mapping、seq lens 每步更新并从 CPU/GPU 同步。
- 内部碎片：每个请求的最后 block 可能未填满。
- Hybrid Attention：滑窗、MLA、Mamba/SSM、跨层不同 block spec 会使 KV manager 更复杂。
- Spec Decode：一轮可能暂存 K 个候选 token 的 KV，接受后 commit、拒绝后回滚/丢弃。

为什么它仍然值得

相比为每条请求预留最大连续 KV，PagedAttention 大幅减少外部碎片和预留浪费，并让 continuous batching、prefix sharing、fork 和动态抢占成为可能。它是“用一点元数据与寻址复杂度，换取系统级显存利用率”。

60 秒口述版

vLLM 先把 KV 显存切成固定 blocks 并一次性预分配。Scheduler 的 KV Manager 在 CPU 管 block id、free list 和 refcount；Block Table 把请求的逻辑块映射到物理块，用于读历史 KV；Slot Mapping 给本轮新 token 指定写入槽位。ModelRunner 每步生成这些 GPU metadata，Attention kernel 直接在 paged pool 上读写。

06

高频问题 60 秒口述速查

面试临场先说结论，再根据追问下钻

01 DP、TP、EP、CP 分别是什么？

DP 复制模型并切请求；TP 切每层矩阵，同一个请求由多卡协同；EP 把专家权重切到不同 rank，token 做两次 A2A；CP 切长上下文/KV。吞吐瓶颈用 DP，权重放不下用 TP，MoE 专家过大用 EP，超长上下文用 CP。

02 TP 和 EP 怎么一起用？

Attention/Dense 层在每个 DP rank 内做 TP，专家层把 DP×TP ranks 组成 EP group。以 TP8 DP2 EP16 为例，总共 16 卡：两个 attention TP8 组，专家跨 16 ranks 分布。请求先绑定一个 DP rank，进专家层后 token 才跨 EP 路由。

03 MoE 两次 A2A 是什么？

第一次 dispatch 把 token 激活按 expert id 发到专家 rank；本地做 grouped GEMM；第二次 combine 把专家输出发回源 rank，inverse permute 后按 gate weight 聚合。通信量近似和 $\text{token} \times \text{top-k} \times \text{hidden} \times \text{dtype}$ 成正比，要发生两遍。

04 通信计算如何 overlap / fusion？

把 token 分 chunk，A2A 异步推进，收到一部分就做专家 GEMM，用双缓冲和独立 stream 隐藏通信。更进一步可融合 router/top-k/permute、quantize+dispatch、grouped GEMM、collective+RMSNorm，甚至 persistent megakernel。最终看 trace，不是异步发起就等于完全隐藏。

05 Executor 与 TP Worker 的关系？

Executor 是跨设备编排抽象，对上是一条 execute_model，对下管理多个 workers。TP8 就通常有 8 个带不同 TP rank 的 workers；每个 worker 对应一张 GPU，并拥有一个 ModelRunner。

06 Prefix Cache 如何复用？

按完整 token block 计算链式 hash，命中后新请求 block table 直接引用已有物理 KV block，refcount 加一，只算未命中的尾部。它只省 prefill；跨实例要用 KV connector 或共享缓存。

07 FP8、FP4、AWQ、GPTQ 怎么区分？

FP8/FP4 是数值格式；AWQ/GPTQ 是 PTQ 算法。AWQ 用激活统计保护显著通道，GPTQ 用近似二阶信息做误差补偿。常见 AWQ/GPTQ 是 W4A16，FP8 常见 W8A8。速度由硬件原生指令和 kernel 决定。

08 在线与离线量化怎么选？

稳定生产优先离线：校准、保存低精度 artifact、固定评测和回滚。在线量化是在加载时转换权重、运行时动态量化激活，适合快速试验和动态模型池，但启动慢、可复现性和后端耦合更差。

09 KV Cache 要不要 FP8？

先算 KV 占比。长上下文/高并发下 KV 是瓶颈时，BF16→FP8 可把 KV 主体约减半，增加容量和降低带宽；要用代表性数据校准 scale，重点回归长上下文、检索、代码和 reasoning。

10 量化方法是否绑定硬件？

算法层可移植，但高性能强绑定：packing、scale、tile、Tensor Core 指令和 kernel 必须匹配。框架负责解析配置、选择 backend、加载/repack 和 layer policy；kernel 负责真实低精度 GEMM/Attention/MoE。

11 完整执行链是什么？

API/AsyncLLM → EngineCore → Scheduler/KV Manager → Executor → Worker → ModelRunner → Model/Kernels/Sampler → detokenize/stream。Scheduler 选工作，Executor 分发，Worker 管设备，ModelRunner 组织 GPU forward。

12 Scheduler 在哪组 batch？

在 EngineCore 的每个 step 做 continuous batching，根据 running/waiting、token budget、sequence budget 和 KV blocks，决定每请求本轮算多少 token；不是在 API 层一次性组批。

13 PD 分离值不值得？

长 prompt、高负载、同时严格要求 TTFT 和 ITL 时值得。P 大 GEMM，D 逐 token 读大 KV，拆开能独立伸缩和隔离干扰；代价是 KV 传输和复杂路由，不天然提升吞吐，要看 SLO goodput。

14 Medusa、EAGLE、MTP？

Medusa 是附加多未来 token heads；EAGLE 是利用 target hidden feature 的轻量 draft；MTP 是 target 原生多 token prediction，不需外部 draft。MTP/Medusa 通常共置，EAGLE 也尽量同节点同卡。

15 Draft Model 怎么部署？

默认每个 DP replica 共置/复制 draft，尽量不跨网络。draft 很小就同卡；显存紧张可同节点独立 GPU；只有大规模和强网络下才考虑独立服务。draft 自身 TP 太大可能失去“便宜”的意义。

16 高并发投机解码会负收益吗？

会。K 增大让 verify 一次处理 K+1 token，高 batch 下可能转为 compute-bound。用 accepted length、committed tokens/forward、draft/verify latency、P99 TPOT 监控，并动态调 K 或高峰关闭。

17 MTP 还是小模型引导？

模型原生支持且生态成熟时优先 MTP；没有 MTP 或 EAGLE/draft 接受率显著更高时用外部 proposer。长期更重要的是动态策略、可观测性和与 serving 的融合。

18 PagedAttention 三句话？

KV 预分配成固定物理 blocks；Block Table 把请求逻辑块映射到物理块用于读；Slot Mapping 给本轮新 token 指定写槽。Scheduler/KV Manager 管 block 生命周期，ModelRunner 生成元数据，kernel 读写。

19 KV 动态分配在哪里？

物理 KV tensor 在 Worker/ModelRunner 启动时一次性建池；CPU 侧 KV Manager 每轮分配/释放 block id、管理 refcount 和 prefix cache；ModelRunner 更新 GPU block table/slot mapping。

参考资料

以下资料用于核对当前 vLLM 主线实现与原始算法。文档更新很快，实际部署请固定版本并以对应 tag 的文档/源码为准。

[R1] vLLM Architecture Overview — https://docs.vllm.ai/en/latest/design/arch_overview/

[R2] vLLM Data Parallel Deployment —
https://docs.vllm.ai/en/latest/serving/data_parallel_deployment/

[R3] vLLM Context Parallel Deployment —
https://docs.vllm.ai/en/latest/serving/context_parallel_deployment/

[R4] vLLM Expert Parallel Deployment —
https://docs.vllm.ai/en/latest/serving/expert_parallel_deployment/

[R5] vLLM Dual Batch Overlap — <https://docs.vllm.ai/en/latest/design/dbo/>

[R6] vLLM Executor API — <https://docs.vllm.ai/en/latest/api/vllm/v1/executor/>

[R7] vLLM Automatic Prefix Caching Design — https://docs.vllm.ai/en/latest/design/prefix_caching/

[R8] vLLM FP8 W8A8 — https://docs.vllm.ai/en/latest/features/quantization/llm_compressor/fp8/

[R9] AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration —
<https://arxiv.org/abs/2306.00978>

[R10] GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers —
<https://arxiv.org/abs/2210.17323>

[R11] vLLM Online Quantization — <https://docs.vllm.ai/en/latest/features/quantization/online/>

[R12] vLLM Quantized KV Cache —
https://docs.vllm.ai/en/latest/features/quantization/quantized_kvcache/

[R13] vLLM Quantization and Hardware Support Matrix —
<https://docs.vllm.ai/en/latest/features/quantization/>

[R14] vLLM Disaggregated Prefilling — https://docs.vllm.ai/en/latest/features/disagg_prefill/

[R15] DistServe: Disaggregating Prefill and Decoding for Goodput-optimized LLM Serving —
<https://arxiv.org/abs/2401.09670>

[R16] Fast Inference from Transformers via Speculative Decoding — <https://arxiv.org/abs/2211.17192>

[R17] vLLM Speculative Decoding — https://docs.vllm.ai/en/latest/features/speculative_decoding/

[R18] vLLM MTP (Multi-Token Prediction) —
https://docs.vllm.ai/en/latest/features/speculative_decoding/mtp/

[R19] Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads —
<https://arxiv.org/abs/2401.10774>

[R20] EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty —

<https://arxiv.org/abs/2401.15077>

[R21] Efficient Memory Management for Large Language Model Serving with PagedAttention (vLLM paper) —

<https://arxiv.org/abs/2309.06180>

[R22] vLLM Paged Attention Design / Kernel Walkthrough —

https://docs.vllm.ai/en/latest/design/paged_attention/

[R23] DeepEP: Communication Library for MoE and Expert Parallelism —

<https://github.com/deepseek-ai/DeepEP>

[R24] LLM Compressor for vLLM — https://docs.vllm.ai/en/latest/features/quantization/llm_compressor/

最后的面试建议

不要把答案背成术语列表

面试官继续追问时，始终回到四个量：显存字节、网络字节、GPU FLOPs、端到端 SLO。说清“数据在哪、谁拥有、何时通信、为什么能隐藏、什么时候会负收益”，比堆框架名更有说服力。